COMSATS University Islamabad

CUI Abbottabad Campus

Midterm Project

AI-Based Network Pathfinding & Attack Simulation System

Project Type:	Algorithm Implementation and Analysis
Course:	Artificial Intelligence (CSC 262)
Instructor:	Zeenat Zulfiqar
Class:	BCS-4B
Due Date:	April 30, 2026

Project Members

Ahsan Ahmad	FA24-BCS-068
Abrar Hussain	FA24-BCS-071

network.json

```
{
"nodes": {
"A": [60, 60],
"B": [200, 60],
"C": [350, 60],
"D": [500, 60],
"E": [130, 190],
"F": [300, 190],
"G": [460, 190],
"H": [60, 320],
"I": [250, 320],
"J": [450, 320]
},
"node_labels": {
"A": "Workstation",
"B": "Switch",
"C": "Router",
"D": "Ext.Firewall",
"E": "Int.PC",
"F": "Server",
"G": "Web Server",
"H": "Admin PC",
"I": "Backup Srv",
"J": "DATABASE"
},
"edges": [
{"from": "A", "to": "B", "cost": 2},
{"from": "A", "to": "E", "cost": 4},
{"from": "B", "to": "C", "cost": 3},
{"from": "B", "to": "F", "cost": 5},
{"from": "C", "to": "D", "cost": 2},
{"from": "C", "to": "G", "cost": 4},
{"from": "D", "to": "G", "cost": 3},
{"from": "E", "to": "H", "cost": 5},
{"from": "E", "to": "F", "cost": 3},
{"from": "F", "to": "I", "cost": 4},
{"from": "F", "to": "G", "cost": 2},
{"from": "G", "to": "J", "cost": 3},
{"from": "H", "to": "I", "cost": 6},
{"from": "I", "to": "J", "cost": 4}
]
}
```

graph.py

```
import json
class Graph:
"""
Represents the network topology as a directed weighted graph.
Nodes = network devices (Workstations, Servers, Firewalls, etc.)
Edges = communication channels with associated vulnerability/risk cost.
"""
```

```
def __init__(self, filename):
with open(filename, "r") as file:
data = json.load(file)
self.nodes = data["nodes"]
self.edges = data["edges"]
self.node_labels = data.get("node_labels", {})
self.blocked_nodes = []
def get_neighbors(self, node):
"""Return list of (neighbor, cost) reachable from node (excluding blocked nodes)."""
neighbors = []
for edge in self.edges:
if edge["from"] == node:
if edge["to"] not in self.blocked_nodes:
neighbors.append((edge["to"], edge["cost"]))
return neighbors
def get_all_nodes(self):
return list(self.nodes.keys())
def block_node(self, node):
"""Defender secures a node – attacker cannot pass through it."""
if node not in self.blocked_nodes:
self.blocked_nodes.append(node)
def unblock_all(self):
"""Remove all defender-secured nodes."""
self.blocked_nodes = []
```

bfs.py

```
from collections import deque
def bfs(graph, start, goal):
"""
Breadth-First Search (Uninformed Search).
Explores nodes level by level (shortest hop-count path).
Does NOT guarantee lowest cost path, but guarantees fewest edges.
Returns:
path (list) - sequence of nodes from start to goal
total_cost (int) - accumulated edge cost of the found path
nodes_expanded (int) - number of nodes fully explored
"""
queue = deque()
queue.append((start, [start], 0))
visited = []
while queue:
current, path, cost = queue.popleft()
if current == goal:
return path, cost, len(visited)
if current not in visited:
visited.append(current)
for neighbor, edge_cost in graph.get_neighbors(current):
if neighbor not in visited:
queue.append((neighbor, path + [neighbor], cost + edge_cost))
return [], 0, len(visited)
```

dfs.py

```
def dfs(graph, start, goal):
```

```
"""
Depth-First Search (Uninformed Search).
Explores as deep as possible along each branch before backtracking.
Does NOT guarantee optimal (lowest cost) path.
Returns:
path (list) - sequence of nodes from start to goal
total_cost (int) - accumulated edge cost of the found path
nodes_expanded (int) - number of nodes fully explored
"""
stack = []
stack.append((start, [start], 0))
visited = []
while stack:
current, path, cost = stack.pop()
if current == goal:
return path, cost, len(visited)
if current not in visited:
visited.append(current)
for neighbor, edge_cost in graph.get_neighbors(current):
if neighbor not in visited:
stack.append((neighbor, path + [neighbor], cost + edge_cost))
return [], 0, len(visited)
```

ucs.py

```
import heapq
def ucs(graph, start, goal):
"""
Uniform Cost Search (Informed Search).
Always expands the node with the lowest cumulative path cost.
GUARANTEES the optimal (lowest cost) path.
Returns:
path (list) - sequence of nodes from start to goal
total_cost (int) - accumulated edge cost of the found path
nodes_expanded (int) - number of nodes fully explored
"""
priority_queue = []
heapq.heappush(priority_queue, (0, start, [start]))
visited = []
while priority_queue:
cost, current, path = heapq.heappop(priority_queue)
if current == goal:
return path, cost, len(visited)
if current not in visited:
visited.append(current)
for neighbor, edge_cost in graph.get_neighbors(current):
if neighbor not in visited:
total_cost = cost + edge_cost
heapq.heappush(priority_queue, (total_cost, neighbor, path + [neighbor]))
return [], 0, len(visited)
```

astar.py

```
import heapq
HEURISTIC = {
```

```
"A": 10,
"B": 8,
"C": 6,
"D": 5,
"E": 7,
"F": 5,
"G": 3,
"H": 8,
"I": 4,
"J": 0,
}
def astar(graph, start, goal):
"""
A* Search Algorithm (Informed Search).
Uses f(n) = g(n) + h(n) to guide search toward goal efficiently.
g(n) = actual cost from start to n
h(n) = heuristic estimate from n to goal (admissible)
GUARANTEES optimal path (because h(n) is admissible).
Returns:
path (list) - sequence of nodes from start to goal
total_cost (int) - accumulated edge cost of the found path
nodes_expanded (int) - number of nodes fully explored
"""
priority_queue = []
heapq.heappush(priority_queue, (0, start, [start], 0))
visited = []
while priority_queue:
f_cost, current, path, g_cost = heapq.heappop(priority_queue)
if current == goal:
return path, g_cost, len(visited)
if current not in visited:
visited.append(current)
for neighbor, edge_cost in graph.get_neighbors(current):
if neighbor not in visited:
new_g = g_cost + edge_cost
h = HEURISTIC.get(neighbor, 0)
new_f = new_g + h
heapq.heappush(priority_queue, (new_f, neighbor, path + [neighbor], new_g))
return [], 0, len(visited)
```

hill_climbing.py

```
HEURISTIC = {
"A": 10, "B": 8, "C": 6, "D": 5, "E": 7,
"F": 5, "G": 3, "H": 8, "I": 4, "J": 0,
}
def hill_climbing(graph, start, goal):
"""
Hill Climbing (Local Search).
At each step, moves to the neighbor with the BEST (lowest) heuristic value.
Greedy - no backtracking. Can get trapped in local maxima / dead ends.
Returns:
path (list) - sequence of nodes from start to goal
total_cost (int) - accumulated edge cost
nodes_expanded (int) - number of nodes evaluated
```

```
"""
current = start
path = [current]
cost = 0
nodes_expanded = 0
visited = [current]
while current != goal:
neighbors = graph.get_neighbors(current)
if not neighbors:
return [], 0, nodes_expanded
best_neighbor = None
best_h = float("inf")
best_edge_cost = 0
for neighbor, edge_cost in neighbors:
h = HEURISTIC.get(neighbor, float("inf"))
if h < best_h:
best_h = h
best_neighbor = neighbor
best_edge_cost = edge_cost
nodes_expanded += 1
if best_neighbor is None or best_neighbor in visited:
return [], 0, nodes_expanded
current = best_neighbor
path.append(current)
cost += best_edge_cost
visited.append(current)
return path, cost, nodes_expanded
def hill_climbing_local_maxima_demo(graph):
"""
Simulates a Hill Climbing failure due to local maxima.
The defender blocks node G, cutting off D's best neighbor.
D (h=5) can only reach C (h=6) - a worse state.
Hill Climbing gets STUCK at D.
"""
start = "D"
goal = "J"
originally_blocked = list(graph.blocked_nodes)
if "G" not in graph.blocked_nodes:
graph.blocked_nodes.append("G")
current = start
path = [current]
visited = [current]
stuck_at = None
while current != goal:
neighbors = graph.get_neighbors(current)
if not neighbors:
stuck_at = current
break
current_h = HEURISTIC.get(current, float("inf"))
best_neighbor = None
best_h = float("inf")
for neighbor, _ in neighbors:
h = HEURISTIC.get(neighbor, float("inf"))
if h < best_h:
best_h = h
```

```
best_neighbor = neighbor
if best_neighbor is None or best_h >= current_h or best_neighbor in visited:
stuck_at = current
break
current = best_neighbor
path.append(current)
visited.append(current)
graph.blocked_nodes = originally_blocked
return path, stuck_at
```

minimax.py

```
NETWORK_VALUES = {
"A": 3,
"B": 5,
"C": 7,
"D": 6,
"E": 4,
"F": 8,
"G": 9,
"H": 2,
"I": 6,
"J": 15,
}
def minimax(node, depth, maximizing_player, graph):
"""
Minimax Algorithm (Adversarial Search).
Two players:
Attacker (Maximizer) - tries to reach high-value nodes.
Defender (Minimizer) - tries to minimize the attacker's gain.
Returns:
best_value (int) - best score achievable from this node
nodes_expanded (int) - total nodes evaluated
"""
counter = [0]
value = _minimax(node, depth, maximizing_player, graph, counter)
return value, counter[0]
def _minimax(node, depth, maximizing_player, graph, counter):
counter[0] += 1
neighbors = graph.get_neighbors(node)
if depth == 0 or not neighbors:
return NETWORK_VALUES.get(node, 0)
if maximizing_player:
best = float("-inf")
for neighbor, _ in neighbors:
value = _minimax(neighbor, depth - 1, False, graph, counter)
best = max(best, value)
return best
else:
best = float("inf")
for neighbor, _ in neighbors:
value = _minimax(neighbor, depth - 1, True, graph, counter)
best = min(best, value)
return best
```

alpha_beta.py

```
from minimax import NETWORK_VALUES
def alpha_beta(node, depth, alpha, beta, maximizing_player, graph):
"""
Alpha-Beta Pruning (Optimized Adversarial Search).
Improves Minimax by pruning branches that cannot affect the final decision.
Alpha = best value the Maximizer (Attacker) can guarantee so far.
Beta = best value the Minimizer (Defender) can guarantee so far.
When beta <= alpha, the remaining branches are pruned.
Returns:
best_value (int) - best score achievable from this node
nodes_expanded (int) - total nodes evaluated (always <= Minimax count)
"""
counter = [0]
value = _alpha_beta(node, depth, alpha, beta, maximizing_player, graph, counter)
return value, counter[0]
def _alpha_beta(node, depth, alpha, beta, maximizing_player, graph, counter):
counter[0] += 1
neighbors = graph.get_neighbors(node)
if depth == 0 or not neighbors:
return NETWORK_VALUES.get(node, 0)
if maximizing_player:
best = float("-inf")
for neighbor, _ in neighbors:
value = _alpha_beta(neighbor, depth - 1, alpha, beta, False, graph, counter)
best = max(best, value)
alpha = max(alpha, best)
if beta <= alpha:
break # Beta cutoff
return best
else:
best = float("inf")
for neighbor, _ in neighbors:
value = _alpha_beta(neighbor, depth - 1, alpha, beta, True, graph, counter)
best = min(best, value)
beta = min(beta, best)
if beta <= alpha:
break # Alpha cutoff
return best
```

main.py

```
from graph import Graph
from ui import NetworkUI
def main():
graph = Graph("network.json")
app = NetworkUI(graph)
app.run()
if __name__ == "__main__":
main()
```

ui.py

```
import tkinter as tk
from tkinter import ttk, messagebox, simpledialog
import time
from bfs import bfs
from dfs import dfs
from ucs import ucs
from astar import astar
from hill_climbing import hill_climbing, hill_climbing_local_maxima_demo
from minimax import minimax
from alpha_beta import alpha_beta
# Color palette
BG = "#0d1117"
PANEL = "#161b22"
ACCENT = "#58a6ff"
SUCCESS = "#3fb950"
DANGER = "#f85149"
WARNING = "#d29922"
PURPLE = "#bc8cff"
TEXT = "#e6edf3"
MUTED = "#8b949e"
CANVAS_BG = "#0d1117"
EDGE_COLOR = "#30363d"
NODE_FILL = "#21262d"
PATH_COLOR = "#58a6ff"
class NetworkUI:
def __init__(self, graph):
self.graph = graph
self.START = "A"
self.GOAL = "J"
self.window = tk.Tk()
self.window.title("AI Network Pathfinding & Attack Simulation")
self.window.geometry("980x780")
self.window.configure(bg=BG)
self.window.resizable(False, False)
self._build_ui()
self.draw_graph()
def _build_ui(self):
tk.Label(
self.window,
text="AI Network Pathfinding & Attack Simulation",
font=("Courier", 15, "bold"),
bg=BG, fg=ACCENT
).pack(pady=(10, 2))
tk.Label(
self.window,
text="Start Node: A (Attacker Entry) | Goal Node: J (DATABASE Target)",
font=("Courier", 9),
bg=BG, fg=MUTED
).pack()
canvas_frame = tk.Frame(self.window, bg=ACCENT, padx=1, pady=1)
canvas_frame.pack(pady=6)
self.canvas = tk.Canvas(
canvas_frame, width=640, height=390,
bg=CANVAS_BG, highlightthickness=0
)
```

```
self.canvas.pack()
ctrl = tk.Frame(self.window, bg=PANEL, pady=8, padx=10)
ctrl.pack(fill="x", padx=20, pady=6)
tk.Label(ctrl, text="Algorithm:", font=("Courier", 10, "bold"),
bg=PANEL, fg=TEXT).grid(row=0, column=0, padx=(0, 4))
self.algo_box = ttk.Combobox(
ctrl,
values=["BFS", "DFS", "UCS", "A*", "Hill Climbing"],
width=14, state="readonly", font=("Courier", 10)
)
self.algo_box.current(0)
self.algo_box.grid(row=0, column=1, padx=4)
def btn(parent, text, cmd, bg_color, row=0, col=2, fg="white"):
b = tk.Button(parent, text=text, command=cmd,
bg=bg_color, fg=fg, activebackground=bg_color,
font=("Courier", 9, "bold"),
relief="flat", padx=10, pady=5, cursor="hand2")
b.grid(row=row, column=col, padx=4)
return b
btn(ctrl, "Run", self.run_algorithm, SUCCESS, col=2)
btn(ctrl, "Block Node", self.block_node, DANGER, col=3)
btn(ctrl, "Unblock All", self.unblock_all, MUTED, col=4)
btn(ctrl, "Compare All", self.compare_all, ACCENT, col=5)
btn(ctrl, "Minimax", self.run_game_algorithms, PURPLE, col=6)
btn(ctrl, "HC Demo", self.hill_climb_demo, WARNING, col=7, fg="black")
self.status = tk.Label(
self.window,
text="Ready. Select an algorithm and click Run.",
font=("Courier", 10), bg=BG, fg=TEXT, wraplength=940, anchor="w"
)
self.status.pack(fill="x", padx=22, pady=(2, 4))
tbl_frame = tk.Frame(self.window, bg=BG)
tbl_frame.pack(fill="x", padx=20, pady=(0, 8))
cols = ("Algorithm", "Discovered Path", "Total Cost", "Nodes Expanded", "Time (ms)")
self.table = ttk.Treeview(tbl_frame, columns=cols, show="headings", height=7)
widths = [110, 320, 80, 120, 90]
for col, w in zip(cols, widths):
self.table.heading(col, text=col)
self.table.column(col, width=w, anchor="center", stretch=False)
self.table.pack(fill="x")
def draw_graph(self, highlight_path=None):
self.canvas.delete("all")
hp = highlight_path or []
for edge in self.graph.edges:
x1, y1 = self.graph.nodes[edge["from"]]
x2, y2 = self.graph.nodes[edge["to"]]
color = EDGE_COLOR if hp else "#3d444d"
self.canvas.create_line(x1, y1, x2, y2, width=2, fill=color,
arrow=tk.LAST, arrowshape=(8, 10, 4))
mx, my = (x1 + x2) / 2, (y1 + y2) / 2
self.canvas.create_text(mx, my, text=str(edge["cost"]),
fill=WARNING, font=("Courier", 8, "bold"))
if len(hp) > 1:
for i in range(len(hp) - 1):
x1, y1 = self.graph.nodes[hp[i]]
```

```
x2, y2 = self.graph.nodes[hp[i + 1]]
self.canvas.create_line(x1, y1, x2, y2, width=4, fill=PATH_COLOR,
arrow=tk.LAST, arrowshape=(10, 12, 5))
R = 22
for node, pos in self.graph.nodes.items():
x, y = pos
if node in self.graph.blocked_nodes:
fill, outline, txt_color = "#3d0c0c", DANGER, DANGER
elif node == self.START:
fill, outline, txt_color = "#0d2e1c", SUCCESS, SUCCESS
elif node == self.GOAL:
fill, outline, txt_color = "#2e0d0d", DANGER, DANGER
elif node in hp:
fill, outline, txt_color = "#0d2040", PATH_COLOR, PATH_COLOR
else:
fill, outline, txt_color = NODE_FILL, "#3d444d", TEXT
self.canvas.create_oval(x-R, y-R, x+R, y+R, fill=fill, outline=outline, width=2)
self.canvas.create_text(x, y-5, text=node, fill=txt_color, font=("Courier", 11, "bold"))
label = self.graph.node_labels.get(node, "")
if label:
self.canvas.create_text(x, y+9, text=label[:10], fill=MUTED, font=("Courier", 6))
def run_algorithm(self):
algo = self.algo_box.get()
t0 = time.time()
if algo == "BFS": path, cost, expanded = bfs(self.graph, self.START, self.GOAL)
elif algo == "DFS": path, cost, expanded = dfs(self.graph, self.START, self.GOAL)
elif algo == "UCS": path, cost, expanded = ucs(self.graph, self.START, self.GOAL)
elif algo == "A*": path, cost, expanded = astar(self.graph, self.START, self.GOAL)
elif algo == "Hill Climbing": path, cost, expanded = hill_climbing(self.graph, self.START,
self.GOAL)
else: return
elapsed = round((time.time() - t0) * 1000, 4)
if path:
self.draw_graph(highlight_path=path)
path_str = " -> ".join(path)
self.status.config(fg=SUCCESS,
text=f"[{algo}] Path: {path_str} | Cost: {cost} | Expanded: {expanded} | Time: {elapsed} ms")
self._add_row(algo, path_str, cost, expanded, elapsed)
else:
self.draw_graph()
self.status.config(fg=DANGER, text=f"[{algo}] No path found.")
def compare_all(self):
for row in self.table.get_children(): self.table.delete(row)
for name, func in [("BFS",bfs),("DFS",dfs),("UCS",ucs),("A*",astar),("Hill
Climbing",hill_climbing)]:
t0 = time.time()
path, cost, expanded = func(self.graph, self.START, self.GOAL)
elapsed = round((time.time() - t0) * 1000, 4)
self._add_row(name, " -> ".join(path) if path else "No path", cost, expanded, elapsed)
self.draw_graph()
self.status.config(fg=ACCENT, text="Comparison complete.")
def run_game_algorithms(self):
t0 = time.time()
mm_val, mm_nodes = minimax(self.START, 3, True, self.graph)
mm_time = round((time.time() - t0) * 1000, 4)
```

```
t0 = time.time()
ab_val, ab_nodes = alpha_beta(self.START, 3, float("-inf"), float("inf"), True, self.graph)
ab_time = round((time.time() - t0) * 1000, 4)
self._add_row("Minimax", "N/A", mm_val, mm_nodes, mm_time)
self._add_row("Alpha-Beta", "N/A", ab_val, ab_nodes, ab_time)
self.status.config(fg=PURPLE,
text=f"Minimax: {mm_nodes} nodes | Alpha-Beta: {ab_nodes} nodes | Pruned: {mm_nodes-ab_nodes}")
def hill_climb_demo(self):
path, stuck_at = hill_climbing_local_maxima_demo(self.graph)
if stuck_at:
self.draw_graph(highlight_path=path)
self.status.config(fg=WARNING,
text=f"[HC Demo] Stuck at '{stuck_at}'. All neighbors have worse heuristic - local maximum!")
messagebox.showinfo("Hill Climbing - Local Maxima",
f"Attacker stuck at node '{stuck_at}'. No better neighbor exists.")
def block_node(self):
node = simpledialog.askstring("Block Node", "Enter node name (A-J):", parent=self.window)
if node:
node = node.strip().upper()
if node in (self.START, self.GOAL):
messagebox.showerror("Error", "Cannot block Start or Goal node.")
elif node in self.graph.nodes:
self.graph.block_node(node)
self.draw_graph()
self.status.config(fg=DANGER, text=f"Node '{node}' blocked.")
else:
messagebox.showerror("Error", f"Node '{node}' not found.")
def unblock_all(self):
self.graph.unblock_all()
self.draw_graph()
self.status.config(fg=MUTED, text="All nodes unblocked.")
def _add_row(self, algo, path_str, cost, expanded, elapsed):
self.table.insert("", "end", values=(algo, path_str, cost, expanded, f"{elapsed} ms"))
def run(self):
self.window.mainloop()
```